

Project 9: Python Web Scraper CI/CD Demo

Overview

This project demonstrates how to deploy a Python data scraping script using Jenkins. The key learning objectives are:

1. **Scheduled cron-based pipeline triggers:** Automating the pipeline to run on a set schedule.
2. **Ephemeral Containers:** Running a script that executes and immediately shuts down (using ``docker run --rm``), unlike long-running web servers.

Prerequisites

Ensure the following are installed and running on your Jenkins server:

- Docker
- Trivy
- Python 3 & pip

Step 1: Network Setup

Create a shared Docker network for your infrastructure and deployments:

```
docker network create jenkins-net
```

Step 2: Infrastructure Setup

Run SonarQube and Jenkins on the ``jenkins-net`` network:

```
# Run SonarQube
```

```
docker run -d --name sonarqube -p 9000:9000 --network jenkins-net  
sonarqube:lts-community
```

```
# Run Jenkins (with Docker socket mounted)
```

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 --network  
jenkins-net -v /var/run/docker.sock:/var/run/docker.sock -v  
jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

Step 3: Jenkins Configuration

1. Log into Jenkins.
2. Install necessary plugins: Docker Pipeline, SonarQube Scanner, Pipeline.
3. Configure SonarQube Server in Manage Jenkins -> System.
4. Configure Global Tools (Sonar Scanner).

Step 4: Pipeline Execution

1. Create a new Pipeline job in Jenkins.
2. Choose "Pipeline script from SCM" or paste the contents of Jenkinsfile.
3. Run the build manually the first time.

Note: Look at the `triggers { cron('H H * * *') }` block in the Jenkinsfile. After the first manual run, Jenkins will automatically schedule this job to run daily!

Step 5: Verification

Review the Jenkins console output for the "Execute Scraping Job" stage. You should see the scraper's output:

- `Successfully scraped example.com!`
- `Page Title: Example Domain`